<Diffusion Model에서의 UNet구조 학습에 관하여>

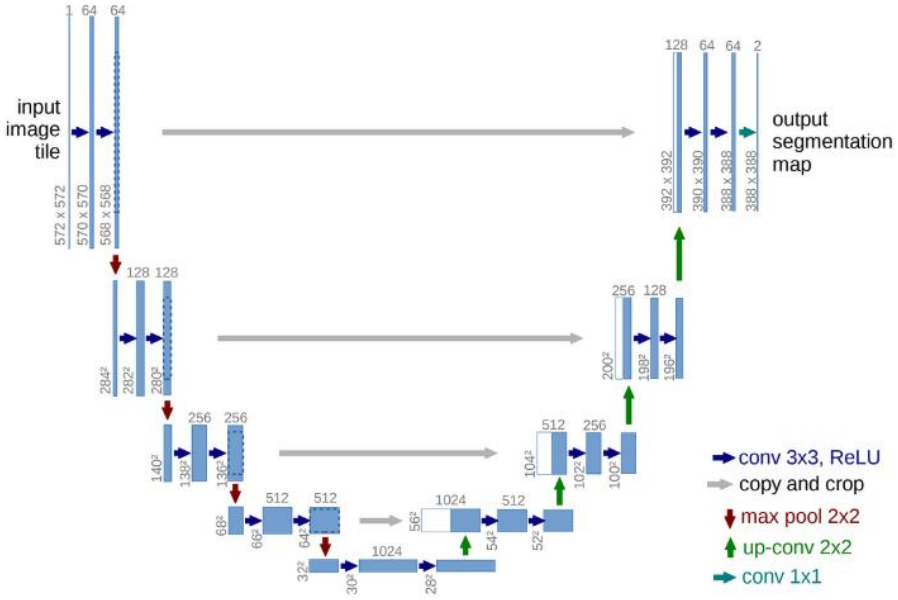
0. Low- & High-level Features

Low level Features : 이미지의 기본적인 요소(edge, corner, texture, color 등)와 같은 저차원적이고 지역적인 정보

High level Features : 이미지의 추상적이고 전역적인 정보(형태, 구조, semantic meaning 등)와 같은 고차원적이고 전역적인 정보

UNet 구조 학습을 통한 추출 : UNet 구조 엔코더 학습의 초기 과정에서는 주로 저해상도 특징(low level features)이 고해상도로 추출되고,
추후 엔코더의 깊은 층을 지나면서 이미지의 복잡한 패턴과 의미 구조(high level features)가 저해상도로 추출된다.
* 정보의 해상도 : 공간적 특징을 세밀하게 제시하면 고해상도 정보 = 즉 이미지의 기본적인(실제) 요소를 다수 포함할 수록 고해상도 정보이므로, low = 고해상도 / high level features = 저해상도 정보인 게 당연.

1. UNet 구조도 개요



UNet 네트워크의 핵심 아이디어는 3 가지로 요약할 수 있다.

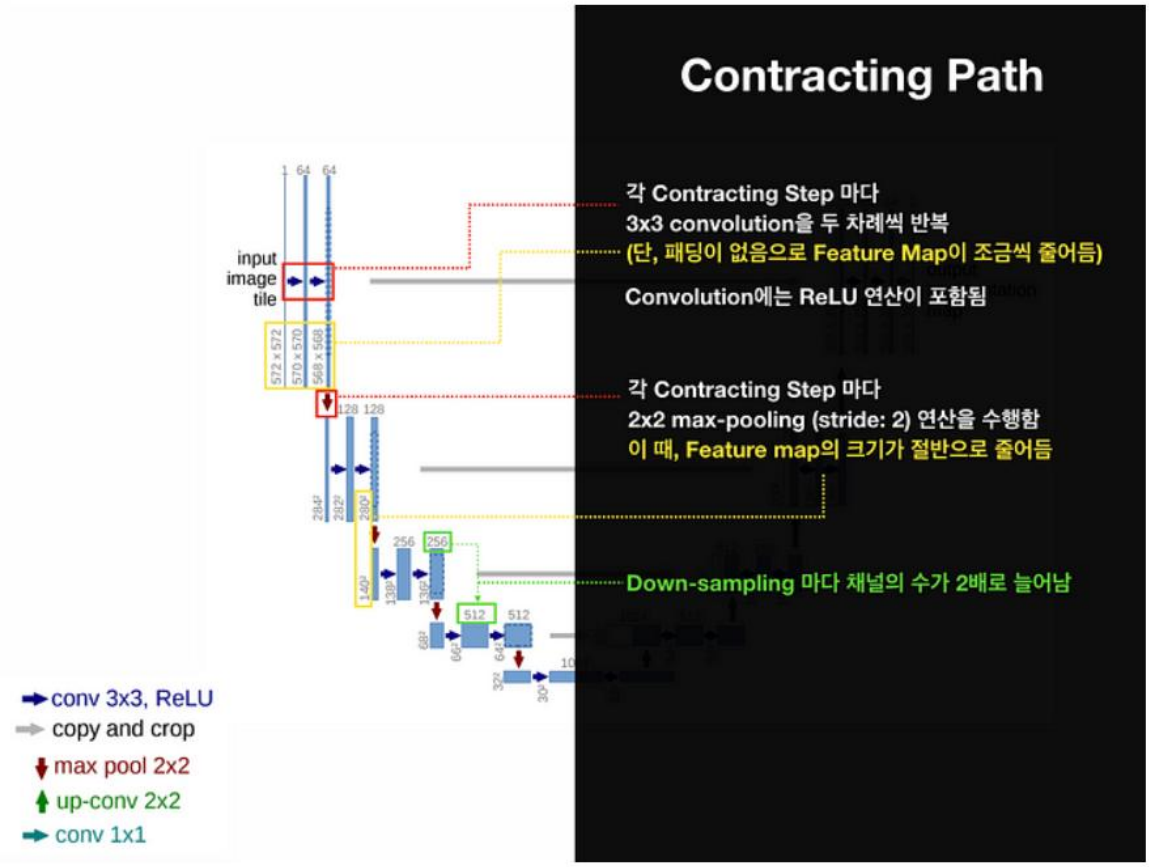
1. 인코더(Contract path)의 피처맵을 디코더 피처맵에 Concat 하여 위치 정보전달
2. 데이터셋의 전처리, 변형(deformation)을 이용하여 데이터 수 증가
3. 테두리(border line)를 더 잘 분할하기 위해 Weight 를 추가한 손실함수(Loss function)

UNet 구조

일반적으로 Semantic Segmentation 모델들은 Down-sampling 을 통해 크기가 줄어들었다가 다시 Up-sampling(데이터 전송 후 복원 과정)을 통해 크기가 늘어나는 구조를 취하는데, UNet 논문에서는 이를 각각 **Contracting Path, Expanding Path** 라고 부른다.

1. Contracting Path(Down-sampling) : 점진적으로 넓은 범위의 이미지 픽셀을 보며 의미정보(Context Information)을 추출
2. Bottle Neck : 수축 경로에서 확장 경로로 전환되는 전환 구간
3. Expanding Path(Up-sampling) : 의미정보를 픽셀 위치정보와 결합(Localization)하여 각 픽셀마다 어떤 객체에 속하는지를 구분

2. Unet – Contracting Path



3×3 convolution 필터를 패딩 없이(stride=1, padding=0) 적용하면, 출력 이미지의 크기는 출력 크기=입력 크기−필터 크기+1 로 작아진다. Stride 는 커널(필터)이 움직이는 간격을 의미하며, 3x3 convolution 에선 1, 2x2 Max Pooling 에선 2 값을 갖는다.

예시 사진을 보면, 입력 이미지(572) – 필터(3) +1(stride) = 570 으로 한 번 3x3 convolution 될 때마다 2 씩 feature map 크기가 소폭 감소한다.

2×2 Max Pooling 은 2×2 영역에서 가장 큰 값 하나만 남기는 연산이다.

stride=2 라서 한 칸씩 겹치지 않고 건너뛰면서 이동한다.

입력 (4×4 예시):

```csharp
[1 3 2 4]
[5 6 7 8]
[9 2 3 1]
[4 5 6 7]
```

2×2 영역 중에서 가장 큰 값만 뽑으면:

- 첫 번째 2×2 영역 → `max(1,3,5,6) = 6`
- 두 번째 2×2 영역 → `max(2,4,7,8) = 8`
- 세 번째 2×2 영역 → `max(9,2,4,5) = 9`
- 네 번째 2×2 영역 → `max(3,1,6,7) = 7`

결과는:

```csharp
[6 8]
[9 7]
```

즉, 4×4 → 2×2로 **해상도가 절반이 됩니다.**

따라서 2x2 Max Pooling 연산 이후에는 feature map 의 크기가 절반으로 감소한다.

채널이란, 하나의 독립적인 정보 층(layer)이다. 하나의 이미지에서 색깔 정보, 모양, 방향, 질감 등을 각각 따로 표현하는 평면들이다.

예시 1: 컬러 이미지의 채널

- 우리가 보는 컬러 이미지 (RGB)는 **3개의 채널**을 가집니다.
  - R (Red)
  - G (Green)
  - B (Blue)

즉, RGB 이미지는 3개의 2D 그리드(행렬)가 쌓인 형태:

$$\text{Shape} = [\text{Height}, \text{Width}, 3]$$

CNN 에서는 convolution 연산을 통해 여러 개의 feature map 을 만들고, 각각의 feature map 이 하나의 채널을 구성한다.

ex) Convolution layer 에서 64 개 필터 사용 → 출력 feature map 은 64 개 채널을 가짐

U-Net 의 Contracting Path 에서는

- 해상도(공간 크기)는 MaxPooling 으로 2 배씩 줄이고
- 표현력(정보량)은 매 sampling 마다 convolution 의 필터 개수를 2 배로 늘려 채널 수를 2 배씩 증량

왜 채널 수를 2 배씩 증량하는가?

✅ 1. 해상도(공간 크기)는 줄어드는데, 표현력은 유지해야 하기 때문

- MaxPooling을 하면 **해상도(공간 차원)**는 줄어듭니다.
  예: `568x568 → 284x284 → 142x142` ...
- 하지만 그만큼 정보가 줄어드는 것을 **보완하기 위해** feature map의 **채널 수**를 늘립니다.

> 즉, 공간 정보를 잃는 대신 채널 수를 늘려 **더 풍부한 표현력**을 확보합니다.

✅ 2. 더 복잡한 특징을 추출할 수 있게 하기 위해

- 처음에는 단순한 edge, corner 등의 low-level 특징을 뽑지만,
- 점점 더 깊이 들어갈수록 객체의 모양, 구조 등 **고수준 정보(high-level semantic feature)**를 뽑게 됩니다.
- 채널 수를 늘려야 더 많은 종류의 feature를 뽑을 수 있습니다.

✅ 3. Decoder(Expanding Path)에서 Skip Connection과 결합을 고려한 구조

- U-Net은 나중에 Contracting Path에서의 feature를 **Skip Connection**을 통해 복원 단계에서 사용합니다.
- 이때 복원 블록(up-conv)에서는 채널 수가 감소하므로, **Contracting에서 채널을 충분히 늘려놔야 대칭 구조가 유지**됩니다.

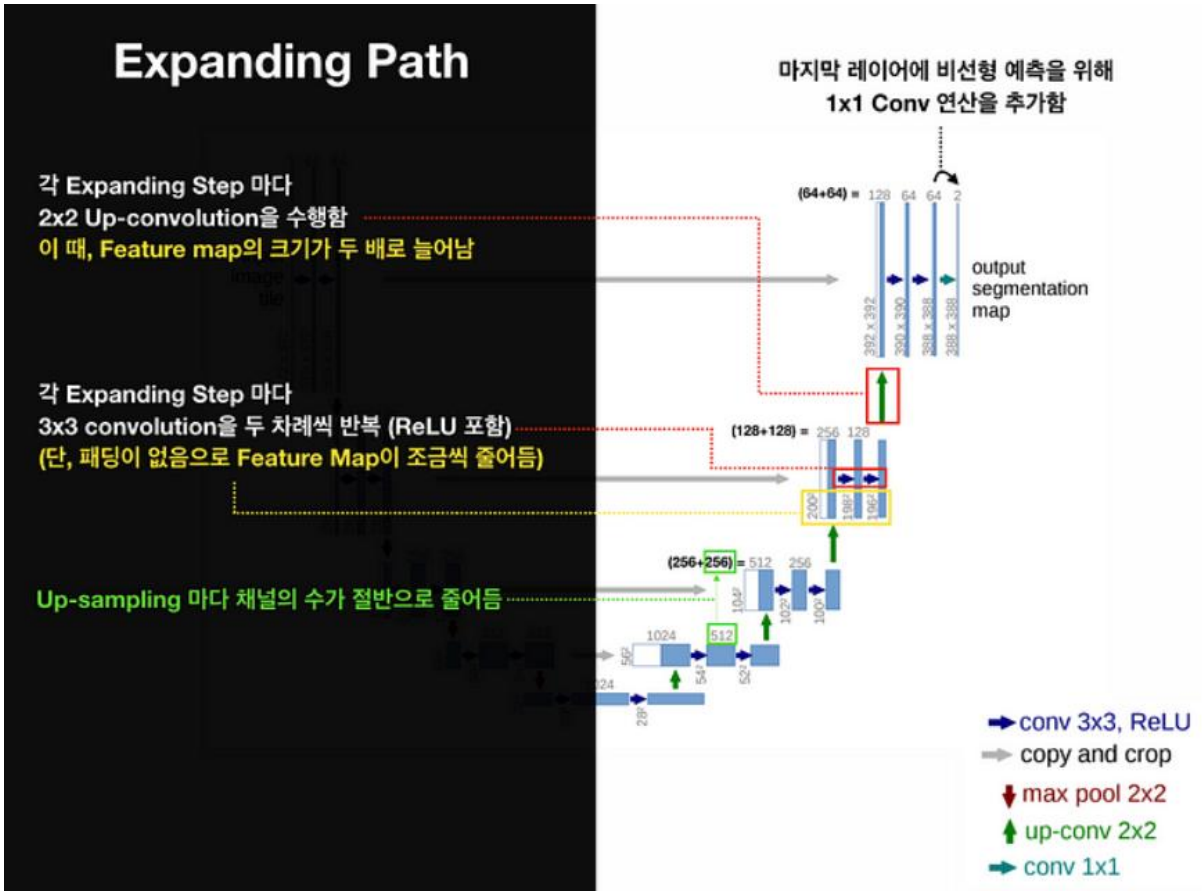
정리하자면,

Contracting Path 의 매 Down-sampling 마다

(Down-sampling 후 feature map 크기) = (sampling 전 feature map 크기 – 4) / 2

(Down-sampling 후 feature map 채널 수) = (sampling 전 feature map 채널 수) * 2

3. Unet – Expanding Path



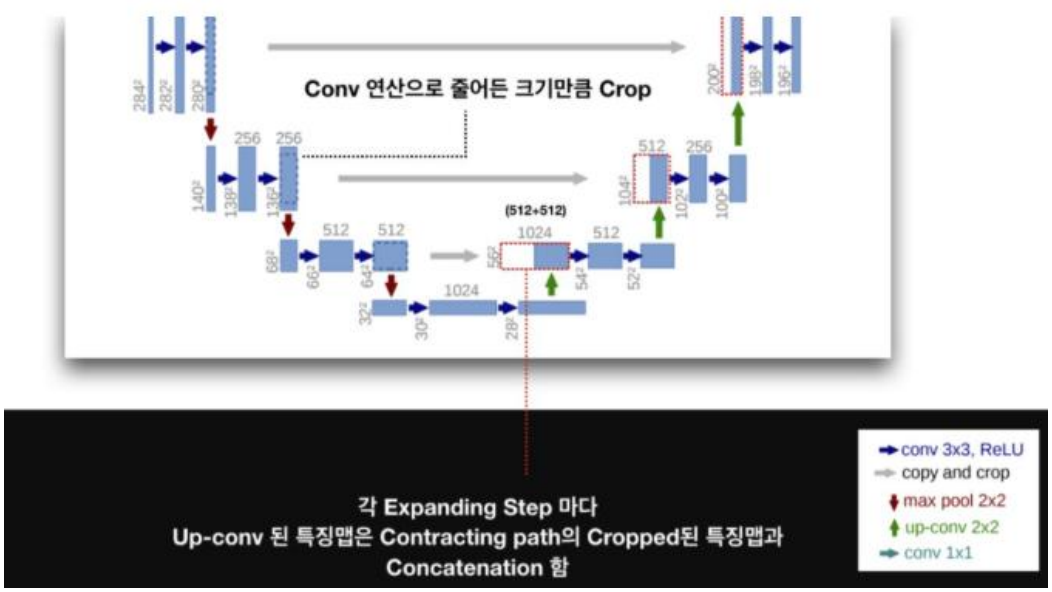
앞서 Down-sampling 마다 채널 수를 2 배 늘린 것을 다시 Up-sampling 에선 convolution 의 필터 수를 2 배로 줄임으로서 채널 수를 원상 복구시킨다. 이 때 2x2 Up convolution 때는 정보량(feature map 의 크기)이 2 배씩 증가하는데, 이는 다양한 방식의 up convolution method 를 이용해 구현한다. 한편 여전히 3x3 convolution 이 두 번 진행되며 padding 이 없으므로 feature map 크기는 2 씩 감소하므로, 정리하자면

Extracting Path 의 매 Up-sampling 마다

(Up-sampling 후 feature map 크기) = (sampling 전 feature map 크기 – 4) * 2

(Up-sampling 후 feature map 채널 수) = (sampling 전 feature map 채널 수) / 2

4. Unet – Skip Connection



Skip Connection 구조란, U-Net 에서 Contracting Path 의 feature map 을 Expanding Path 에 그대로 전달하여 더 정확한 위치 정보를 보존하는 구조이다.

왜 필요한가?

- MaxPooling과 Convolution을 거치면 **해상도는 줄고 추상적 정보는 늘어남**
- 하지만 segmentation 등의 작업에서는 **픽셀 단위의 위치 정보도 매우 중요**
- 따라서 contracting path에서 **중간 feature map을 저장했다가**
  → upsampling한 feature map에 **연결(concatenate)** 하여 **정밀도 보완**

(2) Crop 작업의 이유

> 그림과 설명에서 핵심 포인트:

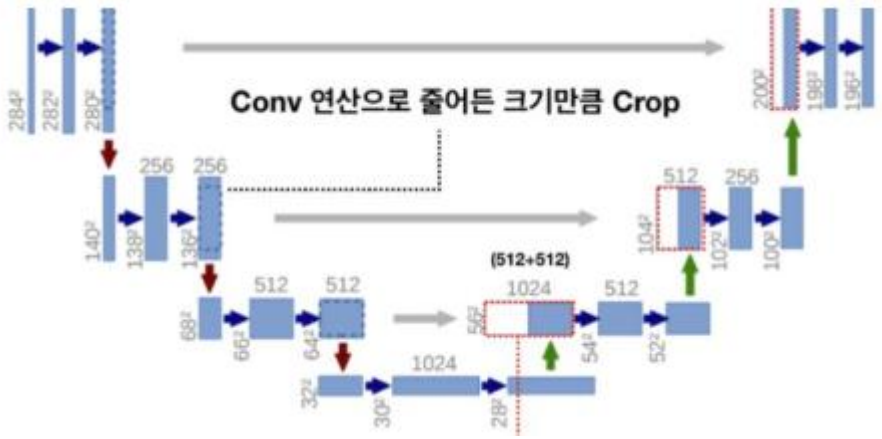
- **Contracting Path의 feature map은 여러 번의 padding 없는 conv 때문에 크기가 더 작음**
- 반면 Expanding Path의 up-conv된 feature map은 보간(interpolation or transposed conv)으로 크기가 복원됨
- **→ 두 feature map의 크기가 달라서 바로 합칠 수 없음**
- 그래서 **Contracting Path의 feature map을 Crop**해서 크기를 맞추고 concatenate 함

🟦 Crop은 파란색 화살표로 시각화되어 있고,
복사(Copy)와 함께 회색 실선으로 concat 연결됨

(3) Concatenation

- 이렇게 맞춘 두 feature map을 **채널 방향으로 합칩니다 (Concatenation)**
- 이후 conv 연산을 통해 정보를 통합합니다.

5. Unet – Bottle Neck

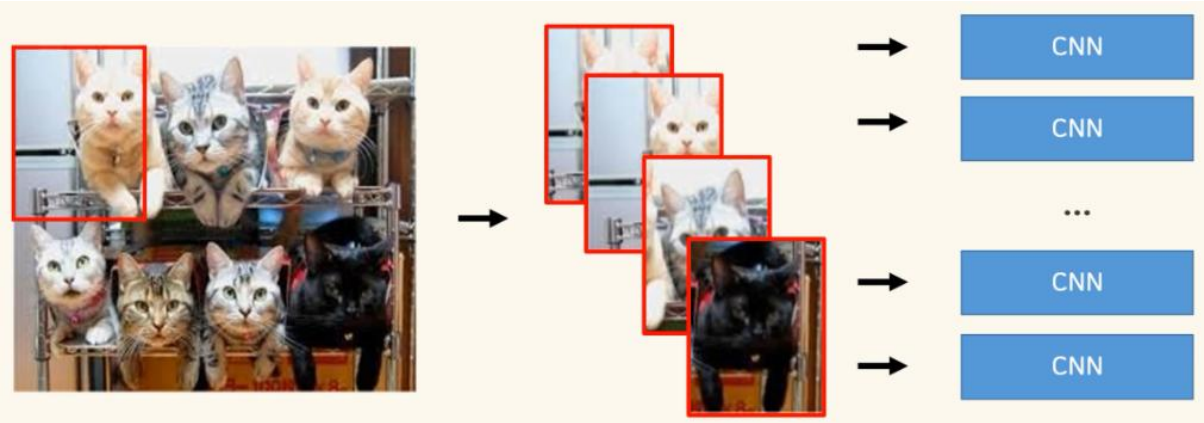


Bottle Neck 은 U-Net 에서 가장 깊은 부분, 즉 해상도는 가장 작고, 채널 수는 가장 큰 구간으로, 압축된 feature map 이 추상적 의미를 가장 많이 담고 있는 부분이다.

딱히 별다른 특별한 연산이 따로 들어가는 것은 아니고 순수하게 저 가장 down-sampling 된 시점을 지칭하는 부분이므로 중요하진 않다.

6. 최초의 UNet 구조 연산 – 1. Overlap-tile strategy

이미지 데이터는 크기가 천차만별이고 사이즈나 해상도가 큰 경우가 많다. 이런 데이터를 사용하려면 크기를 적당히 잘라서 딥러닝 모델에 제시해주어야 한다.



기존의 딥러닝 모델은 sliding window 가 전체 이미지 위로 조금씩 이동하여 해당 영역을 각각의 DNN 에 제시한다. 하지만 이미 사용한 영역의 일부를 다음 순서에서도 다시 훑어보기 때문에 계산량이 많고, 당연히 input image 크기가 클수록 모델의 속도가 크게 저하된다.

이를 UNet 알고리즘 개발자는 연속된 움직임을 보이는 sliding window 로 data 를 뽑는 것이 아닌 주어진 이미지 데이터를 구획화하여 겹치는 부분이 없도록 하는 patch 방식으로 해결하였다.

그렇다면 두 가지 의문점을 쉽게 도출할 수 있다. 첫째, 다양한 크기와 형태의 input image 를 어떻게 UNet 모델에 넣을 일정한 크기의 patch 로 만들 수 있을까? 또한 둘째, UNet 연산 즉 convolution 및 down sampling 으로 압축되며 손실되는 데이터를 어떻게 해결할 수 있을까?

이에 대한 UNet 논문 저자들의 해결법을 소개하기 전에, 해당 논문은 세포의 움직임을 tracking 하는 방법론으로 제시되었음을 밝힌다. 따라서 지금부터 소개할 방법론에 대한 예시는 모두 세포들의 구조와 움직임을 이용하겠다.

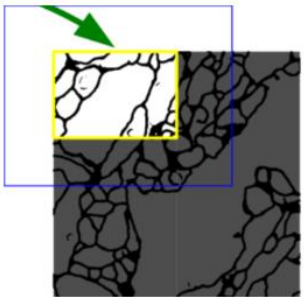
UNet 개발자들은 세포 데이터에 대한 두 가지 가정을 세웠다.

① 일단 작게 자르고 부족한 크기의 이미지는 테두리에 좌우대칭 패딩(Padding)을 하자. 테두리를 좌우대칭을 하는 것이 0 으로 채워넣는 zero padding 보다 실제 모습에 더 가까울 것이다.

② Cell 들은 서로 유사한 형상을 띄므로 기초적인 변형들(deformation)을 주어도 실제로 있을 법한 데이터가 될 것이다.

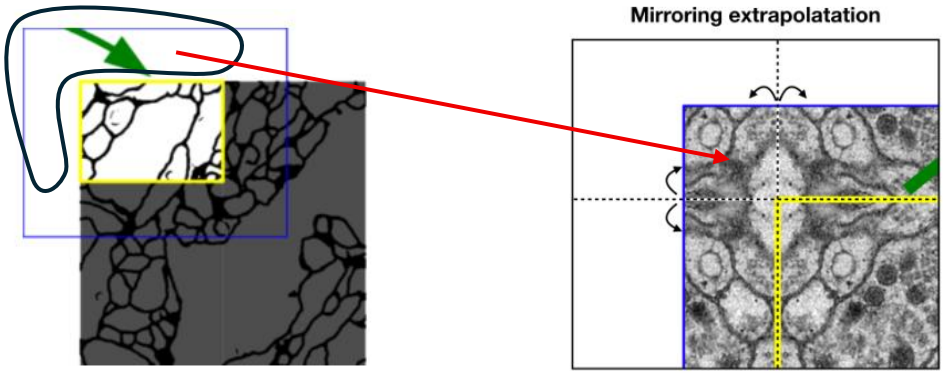
위의 두 가지 가정을 이용하여 데이터 전처리와 변형을 어떻게 구현했는지 알아보자.

7. 최초의 UNet 구조 연산 – 2. Mirroring Extrapolation



논문에 등장하는 UNet 은 388×388 크기의 Segmentation map 을 얻기 위해 572×572 크기의 Input image 가 필요하다. 좌측 그림에서 파란색 patch 를 Input image 로 제시하면, patch 데이터로 UNet 연산을 수행해 결과적으로 노란색 영역의 Segmentation map 이 출력된다. 이미지 크기가 줄어들어 나오는 이유는 앞서 UNet – Contracting Path 에서

봤듯이 padding 없이 Convolution 연산을 반복적으로 수행했기 때문이다.
이 같은 missing data 문제를 해결하기 위해 사용한 것이 mirroring extrapolation 으로, 파란색 박스의 빈 공간을 노란색 영역이 거울에 반사된 형태로 채우는 방식이다.

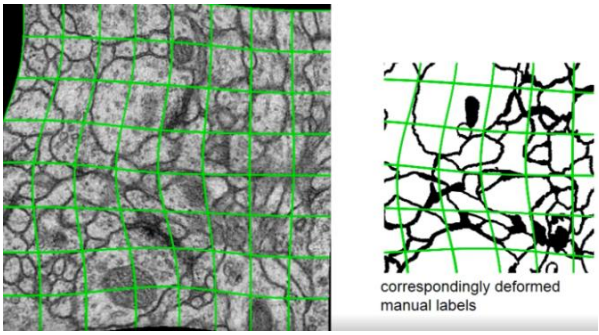


8. 최초의 UNet 구조 연산 – 3. Data Augmentation

Sementic segmentation 은 근본적으로 학습 데이터가 적고, sliding window 로 데이터를 뽑을 때에 비해 patch 로 데이터를 구획한 경우에 데이터량이 더 줄어든다. 따라서 정교한 학습을 위해서는 데이터량을 인위적으로 늘릴 필요가 있고, 이 작업을 data augmentation 이라 부른다.

UNET 저자들이 논문에서 언급한 data augmentation 방식은 총 4 가지이다.

① Shift ② Rotation ③ Gray value ④ Elastic Deformation



그 중 Elastic Deformation 은 Pixel 이 랜덤하게 다른 방향으로 뒤틀리도록 변형하는 방식이다. 좌측 그림을 보면 elastic deformation 을 해도 현실 세계에 있을 법하게 변형되는 것을 볼 수 있다.

9. 최초의 UNet 구조 연산 – 4. Entropy Loss Function

지금까지 기존 딥러닝 알고리즘과 달리 UNet 이 주어진 이미지 데이터에서 input data 를 도출하는 과정과 방법론을 살펴보았다. 이제부터는 input data 를 이용해 어떤 구조의 loss function 으로 네트워크 학습을 구현할 것인지를 배워보자.

논문의 저자들은 loss function 을 픽셀마다 구한 energy function 의 총합으로 구현했다.

* energy function : 딥러닝 모델이 얼마나 잘못된 예측을 했는지 수치로 표현하는 함수
딥러닝에선 이 energy function 의 합을 최소화하는 방향으로 모델을 학습시킨다.

$$E = \sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log(p_{\ell(\mathbf{x})}(\mathbf{x}))$$

이게 UNet 의 loss function 이다.

픽셀의 예측값(Predicition)의 SoftMax p(x)를 구하고 Cross Entropy 를 하였다.

* softmax() : 딥러닝에서 자주 쓰이는 함수로, 분류(classification) 문제에서 모델의 출력을 확률처럼 보이도록 변환하는 함수이다.
더 자세하게는, 여러 개의 출력값(logit)을 받아 총합이 1 이 되도록 정규화(normalization) 해주는 함수이다. 아래와 같은 수식으로 정의된다.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- z_i: i번째 클래스의 출력값 (logit)
- e^{z_i}: 각 출력값에 지수 함수 적용 (항상 양수)
- 분모: 모든 클래스에 대해 지수 함수를 적용한 값들의 합
- 결과적으로 **출력값들이 모두 0~1 사이**이고, **전체 합은 1**

* cross entropy : 딥러닝에서 "cross entropy 를 했다" 라는 건 모델의 예측과 정답(label) 간의 차이를 수치화했다는 뜻으로, 분류 문제의 loss function 으로 자주 사용된다. 예측이 정답에 가까울 수록 cross entropy 값은 감소한다.

정보이론에서 data x 에 대한 정보량은 아래와 같은 식으로 표현된다.

$$\text{Information}(x) = -\log(p(x))$$

- 확률 $p(x)$가 작을수록, 정보량은 커집니다.
- 즉, **"놀라움(surprise)"의 정도**를 나타냅니다.

굳이 log 식으로 표현하는 것은 p(x) 확률의 크기에 따라 정보량이 기하급수적으로 증감하는 점을 반영하기 위함이다.

위의 information 식을 딥러닝에선 cross entropy 함수로 사용한다.

$$\text{CE}(y, \hat{\mathbf{p}}) = -\log(\hat{p}_c)$$

- $\hat{p}_c$: 정답 클래스 c에 대한 softmax 확률

◆ 예시로 이해하기

정답(label)	예측 확률 (Softmax)	Cross Entropy
고양이(1)	[고양이: 0.9, 개: 0.1]	$-\log(0.9) \approx 0.105$ → 🔵 잘 예측함
고양이(1)	[고양이: 0.2, 개: 0.8]	$-\log(0.2) \approx 1.609$ → 🔴 못 예측함

→ 예측이 맞으면 값이 작고, 틀리면 값이 큼

1. 개와 고양이 중 무엇이 정답일지 예측한 결과를 softmax 로 확률화한다.
2. 정답 label "c"의 softmax 확률을 cross entropy 식, -log(확률)에 대입한다.
3. 예측이 제대로 되었을 경우 cross entropy 값이 0 에 가깝게 나온다.

이제 주어진 loss function 을 다시 뜯어보자.

$$\boxed{E = \sum_{x \in \Omega} w(x) \cdot \log(p_{\ell(x)}(x))}$$

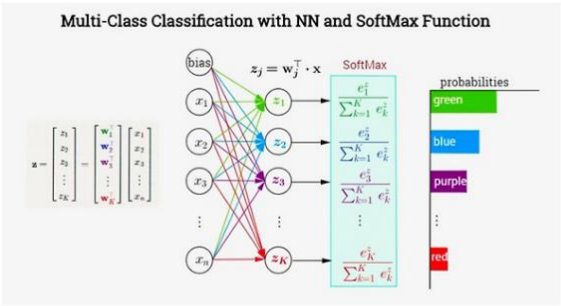
항목	의미
$x \in \Omega$	모든 픽셀 위치 x에 대해
$\ell(x)$	픽셀 x의 **정답 클래스(label)**
$p_{\ell(x)}(x)$	픽셀 x가 **정답 클래스**일 확률 (Softmax 결과)
$\log(p_{\ell(x)}(x))$	→ 바로 이게 Cross Entropy 손실 항
$w(x)$	픽셀별로 다르게 주는 가중치 (Weight map)
E	전체 이미지에 대한 총 loss (에너지 함수)

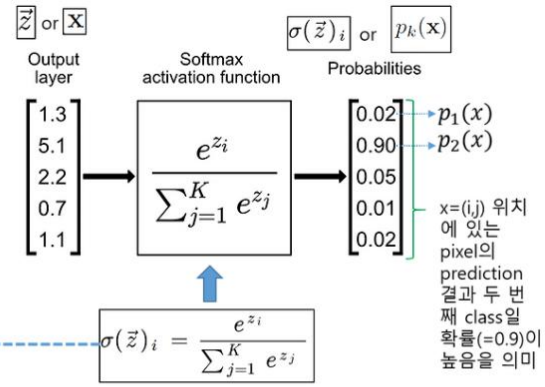
위의 분석에서 알 수 있듯이, **특이하게 cross entropy 에 픽셀 고유의 Weight 를 곱해 픽셀의 loss function 값을 계산**하였다.
여기서 픽셀 고유의 Weight 는 node 와 연결된 Weight 를 의미하는 것이 아니라, 경계선(border)라인에 더 강한 학습을 시키기 위해 gaussian distribution 을 가정하고 경계선의 픽셀에 더 큰 loss weight 를 설정했다.

이러한 경계선에 특화된 loss function 을 통해서 더욱 Segmentation 에 최적화된 학습을 할 수 있었다. 경계선은 두 class 가 만나는 지점으로 예측이 가장 어렵고, 모델이 경계를 모호하게 예측하면 segmentation 품질이 급격히 나빠지기에 매우 중요한 영역이다.

지금까지의 내용을 그림으로 정리해보면 아래와 같다.

1-1) SoftMax





$$p_k(\mathbf{x}) = \exp(a_k(\mathbf{x})) / \left(\sum_{k'=1}^{K} \exp(a_{k'}(\mathbf{x}))\right)$$

K → Number of classes

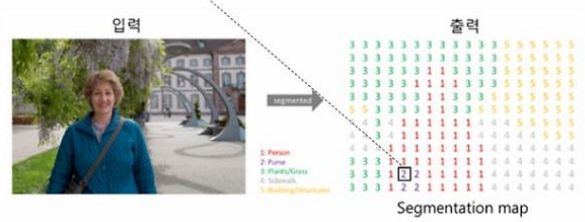
k → k^{th} class

$x \in \Omega \; with \; \Omega \subset Z^2$

$a_k(x)$

Classification softmax와 조금 다른 점은 softmax의 입력변수인 x(=logit)가 2차원 정보를 implicit하게 담고 있다는 점입니다. 그 이유는 segmentation task에서는 pixel 좌표 (i,j) 마다 softmax 값을 구하고 classification (per a pixel) 해줘야 하기 때문입니다. 즉, logit에 해당하는 x는 특정 pixel 좌표의 정보를 represent한다고 할 수 있다.

(11,5)



1-2) Cross-Entropy

$$H(p,q) = -\sum_{x \in \text{classes}} p(x) \log q(x)$$

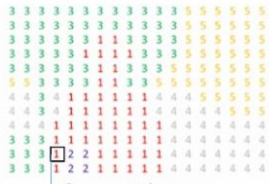
True probability distribution (one-hot)

Your model's predicted probability distribution

Animal	Label
Dog	[1 0 0 0 0]
Fox	[0 1 0 0 0]
Horse	[0 0 1 0 0]
Eagle	[0 0 0 1 0]
Squirrel	[0 0 0 0 1]

$P_1(dog) = 1$
$P_1(fox) = 0$
$P_1(horse) = 0$
$P_1(eagle) = 0$
$P_1(squirrel) = 0$

$P_2(dog) = 0$
$P_2(fox) = 1$
$P_2(horse) = 0$
$P_2(eagle) = 0$
$P_2(squirrel) = 0$



Segmentation map

<Ground truth>
[1, 0, 0, 0, 0]

<Prediction>
[0.98, 0.01, 0, 0, 0.01]

$$H(P_1, Q_1) = -\sum_i P_1(i) \log Q_1(i)$$
$$= -(1 \log 0.98 + 0 \log 0.01 + 0 \log 0 + 0 \log 0 + 0 \log 0.01)$$
$$= -\log 0.98$$
$$\approx 0.02$$

$$Cross\ Entropy = C = \sum_{\mathbf{x} \in \Omega} \log(p_{\ell(\mathbf{x})}(\mathbf{x}))$$

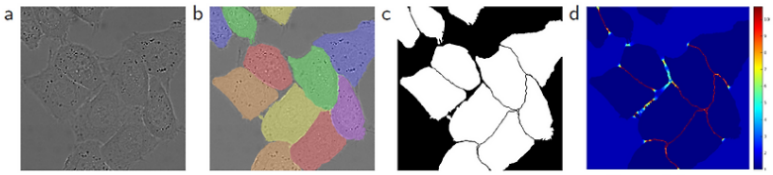
$l: \Omega \rightarrow \{1, \ldots, K\}$ → *the true label of each pixel*

1-3) Weight Map

- 먼저 weight map equation 식 자체에 대한 concept 부터 이해하도록 하겠습니다.

The weight map equation = $w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$ → 해당 픽셀 ($x \subset Z^2$) 의 중요도를 결정해주는 함수

→ 식에 대한 자세한 설명은 다음 그림에서



The weight map

Instance segmentation을 위해 같은 class(=cell)들을 구분해줄 수 있는 경계(=boundary)에 해당하는 pixel을 더 잘 분류 해줄 수 있도록 boundary에 속한 pixel들의 가중치를 더 강하게 주어 더 큰 loss값을 출력해 특별해 해당 pixel(=boundary) 들을 더 잘 분류할 수 있게 학습 시킴

$$Energy\ function = E = \sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log(p_{\ell(\mathbf{x})}(\mathbf{x}))$$

$w_C : \Omega \rightarrow R$ → To balance the class frequencies; Use weighting to balance classes in training data

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$

하나의 이미지에 sky, grass, building, sidewalk가 포함되어 있다고 가정

https://converthere.com/numbers/1.0312e+06-written-out

Name	PixelCount
{'sky'}	3.1485e+05
{'grass'}	1.5979e+05
{'building'}	1.0312e+06
{'sidewalk'}	25313

<부동 소수점 표현>

3.1485e+05 = 3.1485 x 10^5 = 314,850
1.5979e+05 = 1.5979 x 10^5 = 159,790
1.0312e+06 = 1.0312 x 10^6 = 1,031,200

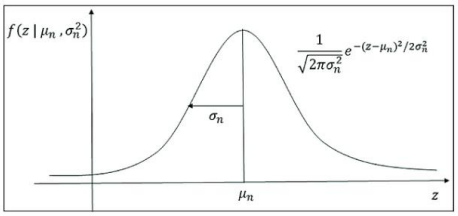
Sum=total Number Of Pixels=1,531,153

$$frequency = \frac{PixelCount}{Total\ Number\ Of\ Pixels}$$

$$w_c(x) = inverse\ frequency = 1/frequency$$

$$w_{sky}(x) = \frac{1531153}{314850} = 4.8632$$
$$w_{grass}(x) = 9.5827$$
$$w_{building}(x) = 1.4848$$
$$w_{sidewalk}(x) = 60.4900$$

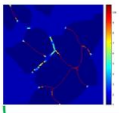
- weight map equation에서 exp 수식 부분 설명
- weight map equation 수식의 오른쪽 수식은 guassian distribution과 유사합니다.



$w_0 = 10$ $\sigma \approx 5$

상수

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$



$e^{-\left(\frac{(1+2)}{(2\cdot(5^2))}\right)} = 0.9417645335842487$

$e^{-\left(\frac{(2+3)}{(2\cdot(5^2))}\right)} = 0.9048374180359595$

결국, cell을 구분 짓는 boundary 영역에 위치한 pixel의 d_1+d_2 값이 작게 나옵니다 → d1+d2의 값이 작을 수록 높은 가중치가 부여됩니다

$w : \Omega \rightarrow R$ → Weight map

$d_1 : \Omega \rightarrow R$ → The distance to the border of the nearest cell: x위치 pixel에서 가장 가까운 세포까지의 거리

$d_2 : \Omega \rightarrow R$ → The distance to the border of the second nearest cell: x위치 pixel에서 두 번째로 가까운 세포까지의 거리

10. UNet 구조 학습의 장단점

- 장점 -

1. 적은 데이터로도 효과적인 학습이 가능
   A. 원 저자들의 실험(의료 영상)에서 매우 적은 수의 학습 데이터(수십 장~수백 장)로도 좋은 성능을 낸다는 것이 알려졌다.
   이는 스킵 연결을 통한 저해상도 + 고해상도 특징의 결합이 효과적으로 이뤄지기 때문이며, 데이터 보강(Data Augmentation) 전략도 함께 사용하기 용이하다.
2. 경계 정보를 잘 유지
   A. 디코더로 올라오는 과정에서 스킵 연결이 세밀한 공간 정보(경계, 모서리 등)를 유지하기 때문에, 오브젝트의 정확한 형태 복원에 강점이 있다.
3. 다양한 해상도 정보의 통합
   A. 엔코더에서 여러 층을 거치며 추출된 다양한 스케일의 특징을 디코더에서 복합적으로 활용한다.
   B. 저해상도에서 얻은 전역적(Contextual) 정보와 고해상도에서 얻은 미세(Micro) 정보가 결합되어, 픽셀 단위의 분류/분할 작업에 적합하다.
4. 확장성 및 변형 구조의 용이성
   A. U-Net 은 비교적 구조가 직관적이어서, 다양한 변형(Attention U-Net, Residual U-Net, Multi-Scale U-Net 등)으로 쉽게 확장 가능하다.
   B. 모델 최적화나 특화된 기능(예: 3D U-Net, 2.5D U-Net)을 적용하기에도 편리하다.

- 단점 –

1. 높은 메모리 사용량
   A. U-Net 은 엔코더뿐 아니라 디코더에서도 대칭적으로 레이어를 쌓고, 스킵 연결에 필요한 중간 피처맵(feature map)도 저장해야 한다.

11. UNet 구조 학습과 Diffusion Model 의 연계

- Diffusion Model 에서 UNet 구조 학습을 자주 사용하는 이유 –

1. 시간 임베딩(temporal embedding)과의 결합 용이
   A. Diffusion 모델에선 시간 단계(시점) t 를 어떻게 네트워크에 주입(conditioning)하느냐가 굉장히 중요하다.
   보통은 sin/cos 주기의 포지셔널 인코딩(PE, positional encoding) 같은 방식을 CNN 에 삽입하는 방식을 사용해왔다.
   그러나 CNN 기반의 UNet 구조는, 블록 단위로 매우 간단하게 time_embedding 을 Residual connection 형태로 주입할 수 있으며, 이로 인해 시간 조건("현재 몇 번째 Diffusion 스텝인가?")을 네트워크가 체계적으로 학습할 수 있다.
   B. Transformer 기반 구조나 다른 복잡한 아키텍처에 비해서, 상대적으로 간단한 모듈 추가만으로 시간 정보를 주입할 수 있다는 것은 이론적·구현적 장점이다.
2. 여러 해상도(멀티 스케일) 조건부 생성에 유리
   A. Stable Diffusion(Latent Diffusion) 등에서 보듯이, 텍스트 조건(text prompt)을 Cross Attention 형태로 추가하고, 이미지 역시 매우 다양한 해상도(512×512, 768×768, etc.)로 다룰 수 있다.
   B. UNet 은 엔코더-디코더 구조상, 각 해상도 레벨에서 Cross Attention 이나 조건부 모듈(Conditional BatchNorm, AdaGN 등)을 손쉽게 붙일 수 있다.
   예컨대 중간 레이어마다 텍스트 임베딩을 어텐션으로 결합함으로써, 텍스트와 이미지의 멀티 스케일 조합을 지원한다.
   C. 위에서 알 수 있듯이 UNet 은 다양한 해상도에서 다양한 타입의 조건(condition)을 섞기에 적합한 유연성을 지닌 구조이다.

(참고 문헌 링크)

https://velog.io/@ad_official/Diffusion-%EC%9D%80-%EC%99%9C-Unet%EC%9D%84-%EC%A2%8B%EC%95%84%ED%95%B4

https://wikidocs.net/148870

https://89douner.tistory.com/297

https://joungheekim.github.io/2020/09/28/paper-review/

https://chatgpt.com/share/686661fc-1ff4-8006-b223-864d43df9828